

System Documentation

Conversational Transfer Functions: experiment setup and web application,
end to end

Kevin Kunkel · Master's thesis · Universität Leipzig

21 September 2026

What this document is docs/rl-paper.typ argues the thesis's claim. This document explains how the whole system is built, so that every number, every screen, and every design decision in it can be traced to the code that produces it. It covers the data pipeline, the transfer-function model, the visibility measurement, the RL formulation and its four checkpoint generations, the evaluation protocol, and the web application — backend, frontend, and voice — in one place.

Contents

1. System overview	3
1.1. Component map	4
2. Data pipeline	5
3. The transfer-function model	6
4. Measuring what a render shows	7
4.1. Validation against real renders	7
4.2. The “other” bucket	7
4.3. The intensity fallback	8
5. Instructions, goals, and scoring	9
5.1. From words to a command	9
5.2. The objective	9
5.3. Instruction kinds and sampling	9
5.4. vessels requires contrast	10
6. The reinforcement-learning formulation	11
6.1. Why one-shot, not ten steps	11
6.2. Observation, action, reward	11
6.3. Hindsight relabelling — implemented, not used	11
6.4. Training	11
6.5. Baselines	12
7. Evolution of the policy: v1 through v4	13
7.1. v2's two bugs, and what fixing them was worth	13
7.2. v3 to v4: fixing a reward-hacking blind spot	13
7.3. The same-day rule-executor fix	14
8. Evaluation protocol	15
9. Results	16
9.1. Limitations, stated plainly	17
10. The web application	18
10.1. Backend architecture	18
10.2. API reference	18
10.3. Frontend	19
10.4. The histogram/transfer-function curve panel	19
10.5. Voice	20
11. Preference collection and RLHF	21
12. Setup and reproduction	22

13. Reading further	23
---------------------------	----

1. System overview

The system turns a spoken or typed instruction about a CT scan into a change to a **transfer function** — the map from Hounsfield intensity to colour and opacity that a volume renderer uses to decide what’s visible. An instruction like “show only the lungs” or “a bit less soft tissue” is parsed into a structured command, turned into a **goal** (a target change in how visible and how bright each anatomical class should be), and answered one of three ways: applied directly by a fixed rule, hill-climbed by search, or proposed in one forward pass by a trained reinforcement-learning policy.

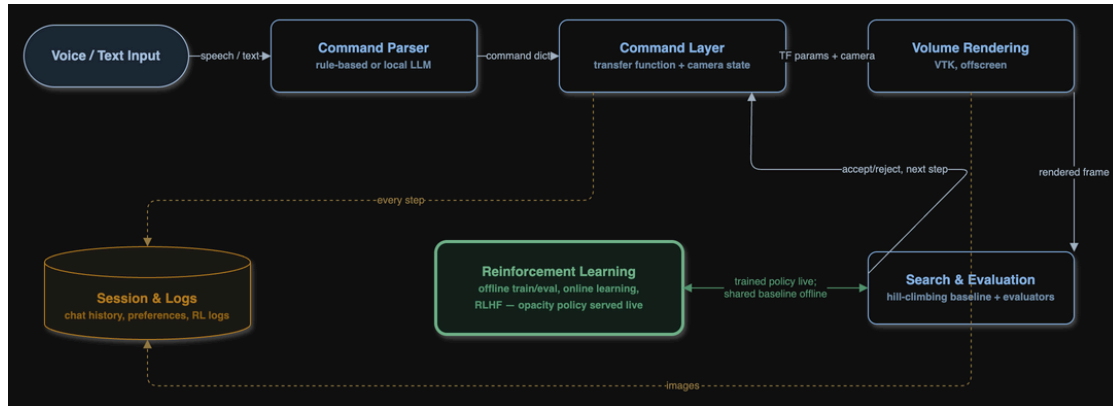


Figure 1: High-level data flow. Voice or text reaches the command parser; the parsed command becomes transfer-function parameters and camera state; volume rendering (VTK) and the cheap visibility estimate both consume those parameters; every step is logged to the session. Search/evaluation and RL sit beside the command layer, not inside the request path, except when the **search** or **policy** answer mode is explicitly chosen.

Caveat: This diagram predates today’s session and is a simplification in two respects worth knowing before reading it literally: “RLHF” and “trained policy live” describe the **intended** end state, not the current one — see Section 11 for the actual status (0 clean preference judgments, reward model not yet trained). And the one-shot policy (Section 6) answers in a single forward pass, not through the iterative accept/reject loop the arrows towards “Search & Evaluation” suggest; that loop describes the hill-climb baselines specifically.

1.1. Component map

Module	Responsibility
<code>server.py</code>	FastAPI backend: session state, rendering, every API route
<code>static/</code>	Frontend: <code>index.html</code> , <code>app.js</code> , <code>viewer.js</code> , <code>style.css</code>
<code>commands.py</code>	Instruction grammar: rule parser, LLM parser, <code>apply_command</code>
<code>asr.py</code>	Voice transcription (faster-whisper)
<code>transfer.py</code>	The 24-parameter transfer-function model and its VTK/opacity math
<code>render.py</code> , <code>views.py</code>	Offscreen VTK rendering, the six standard camera views
<code>visibility.py</code>	Cheap per-class visibility/brightness estimate, validated against VTK
<code>goals.py</code>	Instructions $\rightarrow$ goal vectors; the distance/attainment objective
<code>rl/</code>	Environments (<code>oneshot_env.py</code> , <code>vis_env.py</code>), training, baselines, evaluation
<code>datasets.py</code> , <code>totalseg.py</code>	Volume loading, splits, anatomical labels
<code>collect.py</code>	Blind pairwise preference collection for RLHF
<code>tools/</code>	Data selection, cache building, validation, reports
<code>plots/</code>	Figure generation from <code>rl.vis_eval</code> result files

Table 1: Every module referenced in this document, and where to find it.

2. Data pipeline

30 CT scans from the TotalSegmentator “small” subset (CC-BY-4.0), stratified by body region into 20 training, 4 validation, and 6 test subjects, plus four 3D Slicer sample CTs (ct_chest, ct_skull, ct_cardio, ct_abdomen) held out as an **out-of-source** generalisation set from different scanners.

```
curl -L -o data/Totalsegmentator_dataset_small_v201.zip \
  "https://zenodo.org/records/10047263/files/Totalsegmentator_dataset_small_v201.zip?download=1"
python -m tools.select_totalseg          # select, label, split -> data/
totalseg_manifest.json
python -m tools.build_visibility_cache    # precompute visibility caches
```

Selection is deterministic and committed as data/totalseg_manifest.json, so the splits reproduce without redistributing the scans themselves. Each manifest entry records the subject id, split, SHA-256, whether it’s a contrast-enhanced scan, and which of the five measured anatomical classes are present in its segmentation. Volumes load in canonical RAS orientation (totalseg.load_volume); python -m tools.check_orientation <name> writes projection images to verify it by eye.

Split	Subjects	Contrast	Notes
train	20	3	drives both RL training and instruction sampling
val	4	1	checkpoint selection during training, by median attainment
test	6	1	held out; every number in Section 9 comes from here
out-of-source	4 (Slicer)	1 (ct_cardio)	never trained on; not formally evaluated, see Section 9.1

Table 2: The manifest’s split sizes. “Contrast” counts subjects with contrast: true, the gate that makes vessels a measurable goal (Section 5.4) — only 5 of 30 TotalSegmentator subjects qualify, and only one of the six test subjects (ts_s1379).

Caveat: The out-of-source Slicer CTs have no TotalSegmentator labels at all. visibility.py falls back to a coarse intensity-band classifier for them (Section 4.3), and the policy has never seen them during training. A live check found the policy performing at or below “do nothing” on ct_skull; see Section 9.1.

3. The transfer-function model

A transfer function is 24 floating-point numbers: four peaks, six values each, every value normalised to $[-1, 1]$.

Index (within a peak)	Meaning
0	centre, mapped to Hounsfield units over <code>CENTER_RANGE = (- 1050, 2000)</code>
1	width in HU, mapped over <code>WIDTH_RANGE = (10, 400)</code>
2	height (opacity), unit interval
3–5	colour, RGB, each a unit interval

Table 3: `transfer.PARAMS_PER_PEAK = 6`. A peak’s opacity at a Hounsfield value h is a Gaussian: $\text{height} \times \exp\left(-0.5\left(\frac{h-\text{centre}}{\text{width}}\right)^2\right)$, summed and clipped to $[0, 1]$ across all four peaks; colour is the opacity-weighted blend of each peak’s own colour.

The **anatomical layout** (`transfer.anatomical_params`, what every session and every training episode starts from) places one peak per goal class, centres fixed for the life of the system — no command and no policy action ever moves a centre:

Class	Peak index	Centre (HU)	Width (HU)	Default height
lungs	0	− 800	60	0.05
soft tissue	1	40	80	0.15
vessels	2	300	80	0.30
skeleton	3	900	280	0.60

Table 4: `transfer.ANATOMICAL_*` constants. A command or the policy can change a peak’s height, width, or colour; it can never move its centre or reassign which class a peak represents.

Finding: Skeleton’s width (280 HU) is far wider than the others. Boosting its height still puts real opacity into a neighbour’s HU range — this is precisely the mechanism behind the “show only” haze fixed in Section 7.3, and it is also the reason the **original** pre-RL-v2 version of this project failed (Section 7): “more bone, less spongy” required narrowing the bone peak, and the system had no way to ask for that.

An earlier, retired layout (`transfer.default_params`, before a fix on 2026-09-16) placed peak 0 at fat (− 100 HU) rather than lung parenchyma. `default_params` now simply returns `anatomical_params()`; the distinction is kept in this document only because older screenshots and one retired docstring still describe the band layout.

4. Measuring what a render shows

Direct volume rendering is too expensive to run inside a search loop or an RL step (50–210 ms), and a rendered image alone doesn't say **which tissue** produced which pixel. `visibility.py` estimates both cheaply:

1. Resample the volume once per viewing direction into an 80^3 cube (`GRID_N`) whose first axis runs front to back (`VisibilityModel.from_volume`).
2. Resample the anatomical label volume the same way (nearest-neighbour, not interpolated — interpolating across a tissue boundary would invent a label the volume doesn't actually contain there).
3. For a given transfer function, composite front to back: `weights = alpha`
`#sym.times product(1 - alpha_"before")`, using an opacity/colour lookup table quantised to 256 levels over the same HU range the transfer function is defined on.
4. Sum each class's weight and divide by the ray count for that class's `vis` (share of the final image it contributes); the weight-averaged colour luminance is `bright`; the fraction of rays whose accumulated opacity clears 0.3 is `coverage`.

This runs in about 14 ms on CPU, six standard views per volume (`views.N_VIEWS`).

4.1. Validation against real renders

The reference renders the same transfer function twice — once normally, once with one class's **colour** blacked out and its **opacity** untouched, via VTK label-map masking — and takes the difference in mean luminance as that class's true contribution.

Finding: Deleting a class's voxels instead — the obvious first method — measures something else, because the hole left behind reveals whatever sits behind it. On a bone-dominant transfer function: 0.1481 normally, 0.1065 with skeleton's colour blacked out (its true contribution), 0.1194 with skeleton's voxels **deleted** — higher, because the background showed through. Thin surface tissue anti-correlated with its own visibility under the deletion method.

Class	ts_s1379	ts_s1337	ts_s0454	Status	Notes
skeleton	1.000	0.999	1.000	validated	
organs	0.995	0.994	0.969	validated	
muscle	0.981	0.948	0.999	validated	
vessels	0.992	0.989	(0.001 range)	validated	contrast scans only
lungs	0.89–0.97	—	—	validated	corrected 2026-09-2x, see below

Table 5: Pearson correlation between the estimate (`visibility × brightness`) and the label-masked render reference, 10 transfer functions per class, over the volumes checked.

Caveat: Lungs were previously reported here as unvalidated — “below the renderer's noise floor” — and that was wrong. The reachable-ceiling probe (`solo_max`) that decides whether a class's contribution is measurable at all was built from the **retired** band layout, whose first peak sits at fat (– 100 HU), nowhere near lung parenchyma (– 800 HU). Measured through the correct peak, lungs validate like any other class on five of six checked volumes.

4.2. The “other” bucket

As of today, `features()` also reports `vis["other"]`: the share of the image contributed by voxels the label volume (or the intensity fallback) assigns to **none** of the five classes — fat, connective tissue, partial-volume edges. This did not exist before this session; see Section 7.2 for why it was added and what broke without it.

4.3. The intensity fallback

Any volume that isn't a `ts_*` TotalSegmentator subject (the four out-of-source Slicer CTs, the synthetic phantom) has no anatomical labels at all. `visibility._intensity_class_ids` falls back to coarse Hounsfield bands: ≤ -500 HU is lungs, -30 to 300 HU is organs, $\text{HU} \geq 300$ is skeleton. Muscle and vessels are never populated by this fallback — there is no contrast information to separate them from soft tissue or bone — so `goals.FALLBACK_GOAL_CLASSES` excludes vessels on every volume that isn't a contrast-flagged TotalSegmentator subject.

5. Instructions, goals, and scoring

5.1. From words to a command

Two parsers turn free text into the same structured command shape (`{"target", "attribute", "direction", "strength"}`), or a "compound" list of these):

- **Rule parser** (`commands.parse_command_rule`): a fixed set of regexes over a vocabulary of class synonyms (`CLASS_SYNONYMS`) and verbs (`DIRECT_VERBS` — sharpen/soften/brighten/darken/hide/remove). Deterministic, no external dependency, and the ground truth `rl.baselines.current_executor` mirrors.
- **LLM parser** (`commands.parse_command_llm`): a local Ollama model (qwen2.5:7b by default) given a system prompt describing the exact JSON schema, with a 30 s timeout. Falls back to the rule parser automatically — on a connection error, an invalid schema, or a missing target — so a live demo never hard-fails on an LLM hiccup.

Current status: The viewer’s own default (`static/app.js`, and `server.py`’s `CommandRequest/CompareRequest` defaults) is "llm", not "rule" — voice input benefits most from the LLM’s tolerance for varied phrasing. Internal callers that need determinism (the RL training/eval pipeline, and most of the test suite) explicitly request "rule" instead of relying on a default.

A parsed command becomes a **goal** (`goals.goal_from_command`): for each named class, a requested change in $\log_{10}$ visibility and/or brightness. “Show only” is special-cased (`_goal_show_only`): named classes get +1.0 (10 × more visible, `HIDE_STRENGTH`), every other measurable class gets -1.0.

5.2. The objective

$$D = \sum_i m_i |c_i - d_i| + \kappa \sum_i n_i |b_i - e_i| + \lambda \sum_i (1 - m_i) \max(0, |c_i| - \tau) + \lambda \kappa \sum_i (1 - n_i) \max(0, |b_i| - \tau) + \lambda \max(0, |c_{\text{other}}| - \tau)$$

with $\kappa = 1.5$ (KAPPA), $\lambda = 0.3$ (LAMBDA_KEEP), $\tau = 0.15$ (KEEP_TOLERANCE); c_i, b_i are the achieved $\log_{10}$ -visibility and brightness changes relative to the episode’s start state, d_i/e_i the requested changes, m_i/n_i whether class i ’s visibility/brightness was named at all. **Attainment** is reported throughout: $A = 1 - \frac{D_{\text{final}}}{D_{\text{start}}}$, so 1 means the goal was reached, 0 means no better than doing nothing, negative means worse.

Finding: The keep-tolerance τ is load-bearing, and so — as of today — is its extension to unclassified tissue. Moving one peak always disturbs something adjacent a little, even when the instruction is followed well; a keep term that punished **any** drift, or one that ignored unclassified tissue entirely, would each make leaving the transfer function untouched score better than acting on the instruction. See Section 7.2 for the concrete failure this closed.

5.3. Instruction kinds and sampling

Both training episodes and the held-out evaluation draw from the same grammar (`goals.sample_instruction`, `goals.INSTRUCTION_MIX`):

Kind	Share	Example
relative	40 %	“a bit less soft tissue”
compound	25 %	“more bone, a bit less soft tissue”
show only	15 %	“show only the lungs and skeleton”
absolute	10 %	“high opacity vessels”
brightness	10 %	“brighten the skeleton”

Table 6: The sampled instruction mix every episode (training, validation, and the 200-episode held-out test) is drawn from.

A goal class is only sampled when it's reachable at all: `VISIBLE_CEILING = 0.005` — below half a percent of the image under the best single-peak transfer function, a change is measurable but not something a person could judge, so it's excluded from sampling entirely (this is what silently refuses lung instructions on four of six test subjects, and vessel instructions on all but one).

5.4. **vessels requires contrast**

`goals.goal_classes_for_volume` only offers `vessels` as a goal when the volume is a `ts_* TotalSegmentator` subject **and** its manifest entry has `contrast: true` **and** `"vessels"` appears in its segmentation. None of the picker's six friendly-named demo datasets (`ct_chest`, `ct_skull`, `ct_cardio`, `ct_abdomen`, `mri_head`, `stag_beetle`) satisfy this — `ct_cardio` included, despite its name — because none of them are `TotalSegmentator` subjects at all. In the running viewer, `vessels` is answerable on exactly one dataset: `ts_s1379`.

6. The reinforcement-learning formulation

6.1. Why one-shot, not ten steps

The first formulation gave the policy ten steps of ± 0.1 per parameter, rewarding the drop in goal distance at each step. It never learned, and training longer made it **worse** — flat to degrading across every setting tried (reward scale, exploration, gradient steps per env step).

Finding: The diagnosis is structural. A ten-step policy must commit to each change **blind**; the hill-climber it's measured against may try a change, measure it, and reject it. It was being asked to learn a search procedure — a strictly harder problem than the mapping the thesis actually asks about, and one a 10-evaluation search already does well. Emitting the whole transfer function in one forward pass removes the search from the policy's job and leaves only the mapping.

6.2. Observation, action, reward

`rl.oneshot_env.OneShotEnv`, one step per episode:

Component	Definition
Observation (57)	goal vector (16) + intensity histogram (16) + per-class $\log_{10}$ visibility (4) + brightness (4) + $\log_{10}$ reachable ceiling (4, <code>solo_max</code>) + current controllable parameters (12) + coverage (1)
Action (12)	the new value — not a delta — of each controllable group: height, width, and colour per peak, in $[-1, 1]$; centres are never controllable
Reward	attainment, clipped to $[-1, 1]$, minus 1.0 if the result shows effectively nothing (<code>goals.is_useless</code> : coverage $< 1\%$ or total visibility $< 0.1\%$)
Episode	exactly one step

Table 7: The one-shot environment every checkpoint from v1 onward trains against.

Reset noise: each controllable group gets uniform ± 0.3 noise (`START_NOISE`) added to the anatomical default before the episode starts, so the policy sees more than one exact starting point per volume.

6.3. Hindsight relabelling — implemented, not used

`OneShotEnv` supports `hindsight_ratio`: with that probability, an episode draws a **reachable** random action first, measures what it achieved, and uses that as the goal instead of a sampled instruction — guaranteeing every such episode has a demonstrated solution. The noise scale (0.25 controllable units) is calibrated against 200 synthetic episodes to land in the same delta range real instructions ask for.

Caveat: `hindsight_ratio` defaults to `0.0`, and the actual training invocation used for every checkpoint in Section 7 does not override it. This machinery is fully built and calibrated but has never been turned on in a run that produced a reported number.

6.4. Training

```
python -m rl.oneshot_train --timesteps 150000 --seed {0,1,2} \
  --eval-interval 10000 --out out/rl_v2/oneshot_v{N}_seed{seed}
```

SAC (stable_baselines3.SAC, MlpPolicy), one override from library defaults: `learning_starts=500` (vs. 100), giving the replay buffer a few more random episodes before learning starts. Every `eval-interval` steps: 40 fixed validation episodes, deterministic action, logged to `eval_progress.csv`; the checkpoint is kept as `best.zip` by **median** attainment, not mean — a single very-negative episode on an easy goal (attainment is unbounded below) can otherwise swing which checkpoint looks best.

6.5. Baselines

Baseline	Mechanism	Evaluations
B0 do nothing	returns the start state unchanged	0
B1 rule executor	commands-style rule: named class(es) to height 0.7 (width capped to 150 HU as of today, Section 7.3 for <code>show_only</code>), one asymptotic step per mentioned class otherwise	0
B2 random	10 random ± 0.1 steps, ignoring the instruction entirely – the floor	0
B3/B4 hill-climb	coordinate search directly minimising the objective (Section 5.2): try $\pm$ step on each of 12 dims, keep improvements, halve the step when a sweep finds none	10 / 200
B5 occlusion rule	B1, then crushes every peak not named or increasing to height 0.02	0

Table 8: Every baseline `rl.baselines` implements. B3/B4 have **oracle access** to the exact scoring function at inference time; the policy and B0/B1/B2/B5 never see `goals.distance` outside training.

7. Evolution of the policy: v1 through v4

The one-shot formulation went through four checkpoint generations. Each was retrained from scratch, not fine-tuned from the previous one, so a later generation’s numbers never carry an earlier bug’s habits forward.

Version	Checkpoints	What changed
v1	oneshot_seed{0,1}	First one-shot checkpoint, replacing the ten-step formulation. No held-out evaluation was ever run against it — superseded within a day by v2. A 53-value observation (no reachable-ceiling channel).
v2	oneshot_v2_seed{0,1,2}	Added the reachable-ceiling channel to the observation (57 values). Evaluated, but on two undetected bugs, below.
v3	oneshot_v3_seed{0,1,2}	Both bugs fixed. Headline checkpoint from 2026-09-17 until today.
v4	oneshot_v4_seed{0,1,2}	goals.distance gained the “other” keep term (Section 7.2) — a gap v1 through v3 all shared. Current , shipped in the viewer as seed 2.

Table 9: Directory names under out/r1_v2/ match this table’s version column exactly.

7.1. v2’s two bugs, and what fixing them was worth

v2’s reachable-ceiling probe (solo_max) was built from default_params() — the retired band layout — so peak 0 was probed as fat, not lungs, the same mismeasurement described in Section 4.1. Separately, CONTROLLABLE groups a peak’s (r, g, b) into one value (its mean); the one-shot policy **set** that group instead of adding a delta to it, which wrote the same scalar into all three channels — every policy render came out grey, while every baseline stayed coloured, so a rater could tell which candidate was the policy’s on sight. Six preference judgments collected under this bug are archived (out/vis_preferences_pre_colour_fix.jsonl) rather than used.

Statistic	v2	v3	Gap
median of seed medians	+0.247	+0.275	+0.028
median of seed-averaged per-episode	+0.201	+0.286	+0.086

Table 10: Same 200 episodes, two aggregations. Paired Wilcoxon on the seed-averaged per-episode values: $p = 0.0064$, v3 ahead on 59 % of episodes. The gain concentrates in **absolute** instructions (+0.122 of +0.038 average kind-level gain) — exactly the kind whose target is a share of the reachable ceiling, the channel that was repaired.

Caveat: A published intermediate figure (+0.169 for v3) was a measurement accident, not a defect in the policy: the evaluation job was started before the ceiling fix landed and finished six hours after, so it scored with the code the process had loaded at import time, not the code on disk. provenance.py now records the git commit and a fingerprint of the imported scoring modules in every result file specifically so this can’t recur silently.

7.2. v3 to v4: fixing a reward-hacking blind spot

visibility.py measured five classes, but every voxel belonging to none of them was invisible to the objective — not measured, not penalised, not present anywhere in goals.distance.

Finding: A transfer function could satisfy “show only X” by rendering an opaque wall of unclassified tissue instead of X. Driving hill_climb (200 evaluations) on “show only bones” reached **full frame coverage** while every one of the five labelled classes read below 0.001 of the image — 99 % of the

frame was material the objective could not see, and the search still scored it as a strong answer, because nothing charged for it.

Fix: `visibility.py`'s `features()` gained an "other" bucket (Section 4.2); `goals.distance` charges for it under the same keep-tolerance any unmentioned class already gets (Section 5.2). Verified in isolation before retraining — new unit tests confirm a transfer function that hides a class behind unclassified material now scores strictly worse than hiding it cleanly.

Method	v3	v4	Improved
hill-climb (thorough)	+0.730	+0.692	100 % → 100 %
policy + 3 refinements	+0.316	+0.371	76 % → 80 %
policy alone	+0.275	+0.331	73 % → 78 %
hill-climb (cheap)	+0.263	+0.258	93 % → 93 %
rule executor	− 0.022	− 0.022	37 % → 37 %
occlusion heuristic	− 0.191	− 0.198	30 % → 30 %

Table 11: Median of seed medians, same 200-episode protocol. Thorough search's own score **drops** — part of its old advantage was the same exploit, now correctly discounted rather than rewarded.

Caveat: The fix did not resolve the failure mode it explains. Re-running “show only bones” on `ts_s0477` with the strongest new seed still reaches only 0.18 % skeleton visibility against 60 % “other” — attainment rose (+0.246 → +0.289) from cleaner suppression of lungs/soft tissue, not from the policy learning to raise the named class. `show_only`'s own median-of-medians (+0.166) sits at or slightly below the old single-seed figure (+0.20). One retrain removed the wrong incentive; it did not teach the right behaviour.

7.3. The same-day rule-executor fix

A wide peak undermines its own “show only”: skeleton's 280 HU width still carries real opacity into soft tissue's HU range, so boosting skeleton's height alone can **increase** soft-tissue visibility, the opposite of what “show only bones” asked for. `commands.apply_command` and `rl.baselines.current_executor` now also cap the shown peak's width to 150 HU (`transfer.SHOW_ONLY_MAX_WIDTH_HU`), capping only — never widening an already-narrow peak. On the case that exposed it: soft-tissue haze fell 124× (3.98 % → 0.03 %) for a 12 % cost to skeleton's own visibility, turning a clear regression (− 0.448) into a clear win (+0.148). B1's **aggregate** 200-episode score barely moved (− 0.022 → − 0.0225) — which class is asked for and how bright it started matters more to that number than this fix does.

8. Evaluation protocol

`rl.vis_eval.fixed_episodes` materialises a deterministic list of `(volume, start_params, instruction)` triples — the same list every time it’s called with the same `split/seed/count`, which is what makes runs across policies and seeds comparable. Every held-out number in this document and in `docs/rl-paper.typ` is 200 such episodes on the `test` split, seed 0.

Every baseline and the policy score the identical episodes through the identical `goals.attainment` call (Section 5.2) — an arm’s mechanism differs, its scoring never does. Significance is a paired Wilcoxon signed-rank test with the normal approximation and continuity correction (`rl.vis_eval.wilcoxon`) — hand-implemented, `scipy` is not installed in this environment, verified against a hand-computed worked example in its own docstring and a unit test.

9. Results

Method	Evaluations	Median attainment	Improved
hill-climb (thorough)	200	+0.692	100 %
policy + 3 refinements	4	+0.371	80 %
policy alone	0	+0.331	78 %
hill-climb (cheap)	10	+0.258	93 %
do nothing	0	0.000	—
rule executor	0	− 0.022	37 %
random	0	− 0.040	39 %
occlusion heuristic	0	− 0.198	30 %

Table 12: Current headline: v4, 200 held-out instructions, six unseen patients, median of three seeds’ medians.

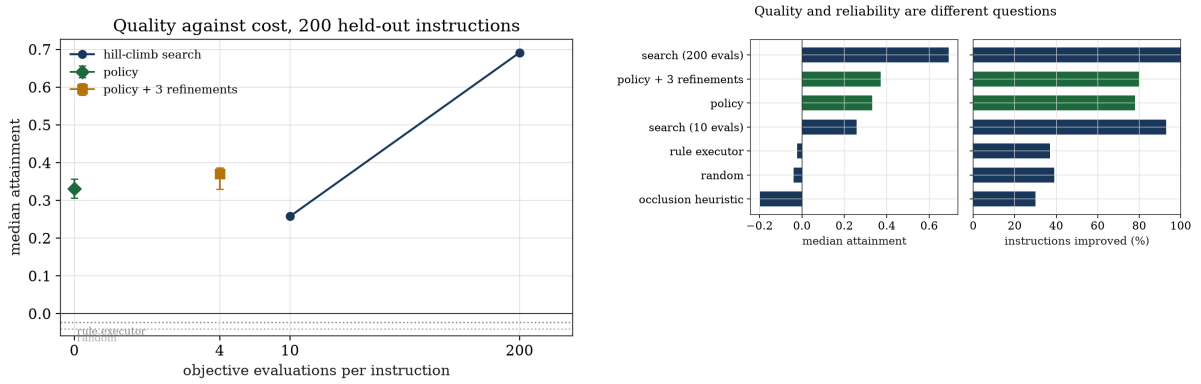


Figure 2: Quality against cost, and quality against reliability — the same table, plotted two ways. Generated by `plots.held_out_results` directly from the `rl.vis_eval` result files; nothing in these figures recomputes a score.

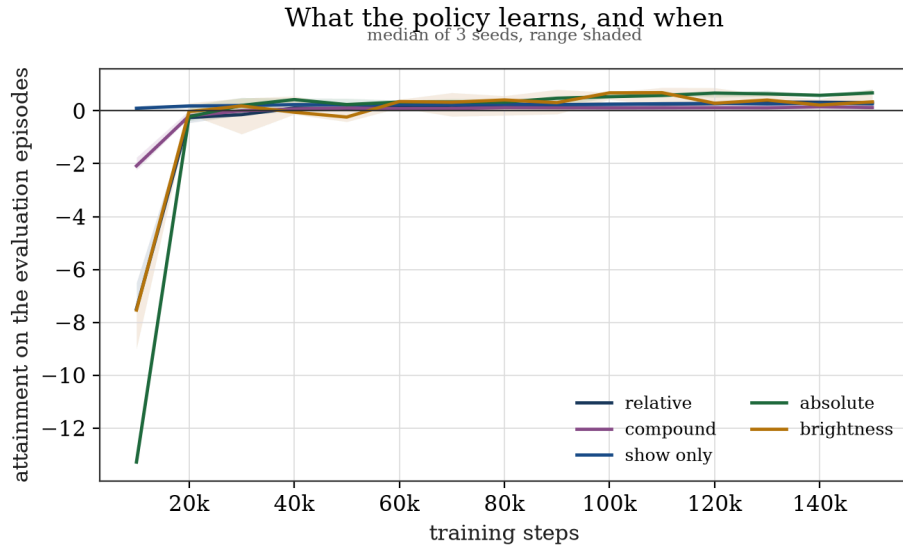


Figure 3: Attainment per instruction kind over training, median of the three v4 seeds, range shaded. Absolute, brightness and relative plateau comfortably positive; compound and show-only plateau near zero.

Instruction kind	Median of medians, v4
absolute	+0.511
brightness	+0.561
relative	+0.354
show only	+0.166
compound	+0.117

Table 13: Per-kind breakdown. Compound moved from the v3 figure of +0.09 to +0.117 — still weak, but not nothing.

9.1. Limitations, stated plainly

- **show_only is not solved for the policy** (Section 7.2) — the reward no longer rewards the wrong thing, but nothing has yet taught the policy the right thing.
- **hill_climb’s coordinate descent finds the same “suppress everything, light up unclassified tissue” local optimum** on show_only, from the default start and from randomised starts, even under the corrected objective. The fix removes the wrong incentive; it does not guarantee a greedy local search finds the right one. Left unaddressed deliberately — changing search behaviour would need its own budget re-validation.
- **Vessels answer on exactly one dataset** (ts_s1379, Section 5.4).
- **Out-of-source generalisation has never been formally evaluated**, despite datasets.OUT_OF_SOURCE_CT and rl.vis_eval --split out_of_source existing for exactly that.
- **Compound instructions remain the second-clearest capability gap** (+0.117) after show-only.

10. The web application

10.1. Backend architecture

`server.py` is a single FastAPI application plus one included router (`collect.router`). All mutable state lives in one `Session` object — module-level, not per-request — persisted to `out/ui_session.json` after every command so a server restart resumes where the session left off. `Session.history` is an append-only list of **steps**; `cursor` is the current position, so back/forward are just cursor moves, not re-computation.

Field	What it holds
<code>id, timestamp</code>	step identity
<code>cmd_text, cmd_dict</code>	what was said/typed, and its parsed form
<code>params</code>	the 24-float transfer function, after this step
<code>camera</code>	azimuth/elevation/zoom
<code>image_b64, image_path</code>	the rendered PNG, inline and on disk
<code>masses</code>	legacy opacity-mass readout over the retired HU bands
<code>class_visibility</code>	per-goal-class visibility share — the policy’s own metric
<code>curve, histogram</code>	the transfer-function curve and volume histogram, added today (Section 10.4)
<code>features</code>	coverage/brightness features straight off the rendered image
<code>mode</code>	“exact” “search” “policy” “camera”
<code>parser_requested/used/fallback</code>	which parser was asked for vs. what actually ran

Table 14: Every field `server._render_step` writes onto a step (`server.py`).

10.2. API reference

Route	Method	Purpose
<code>/</code>	GET	serves <code>static/index.html</code>
<code>/api/state</code>	GET	current step + cursor/total
<code>/api/back, /api/forward</code>	POST	move the history cursor
<code>/api/datasets</code>	GET	list loadable dataset names
<code>/api/datasets/{name}/metadata</code>	GET	dimensions/spacing for the local <code>vtk.js</code> viewer
<code>/api/datasets/{name}/chunks/{i}</code>	GET	chunked raw volume bytes for the local viewer
<code>/api/commands</code>	GET	the command reference (<code>COMMANDS.md</code> ’s source)
<code>/api/dataset</code>	POST	switch the active dataset
<code>/api/command</code>	POST	parse + apply/search/answer one instruction
<code>/api/compare</code>	POST	answer one instruction four ways without advancing the session
<code>/api/compare/sweep</code>	POST	the same four ways, across many sampled instructions
<code>/api/scenes/transition</code>	POST	append a de-duplicated scene-transition record
<code>/api/transcribe</code>	POST	voice → text via <code>faster-whisper</code>
<code>/collect</code>	GET	the blind preference-collection page
<code>/api/collect/next</code>	POST	the next pair to judge
<code>/api/collect/judge</code>	POST	record a judgment

Table 15: The complete route table, `server.py` + `collect.py`.

`/api/command` and `/api/compare` both accept a `parser` field (default `"llm"`, Section 5.1) and a `mode` field (`"exact" | "search" | "policy"`). `/api/compare` parses the instruction **once** and builds one goal from it, so all four arms answer the identical parsed command — a bad parse then makes every arm wrong together, read as a parsing problem rather than a policy problem, not four independently-wrong answers.

10.3. Frontend

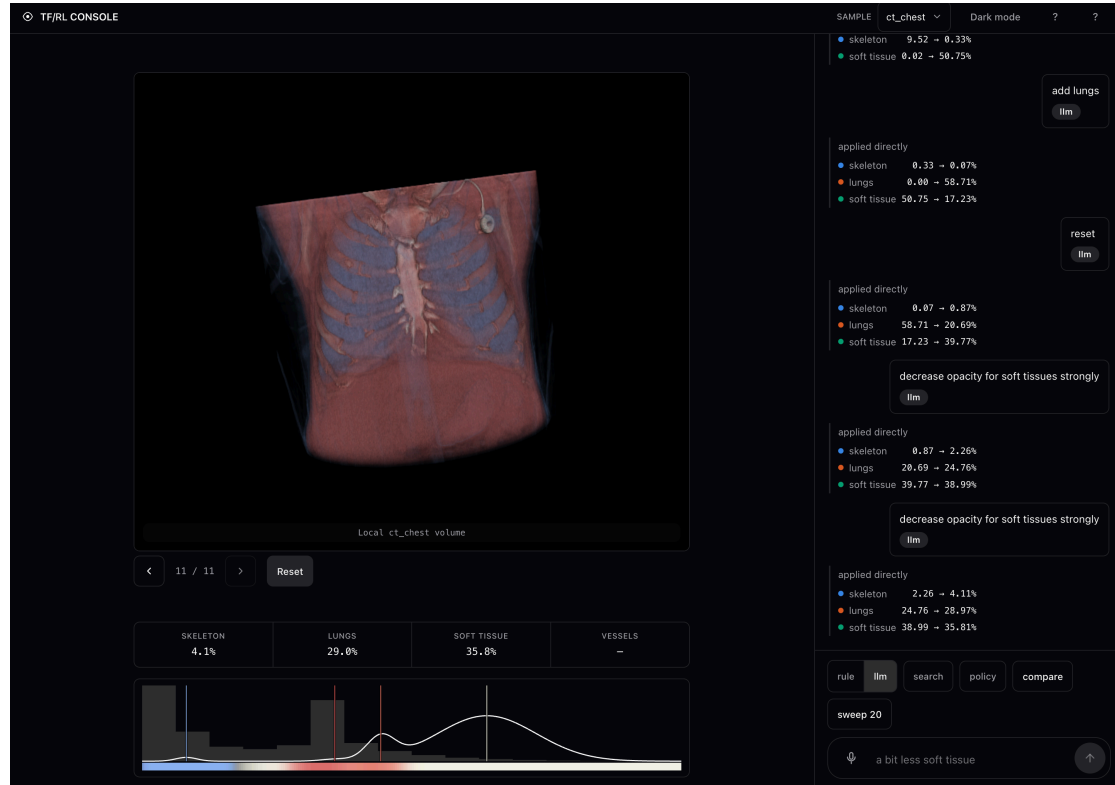


Figure 4: The viewer mid-session: a render, the instruction history with each command’s per-class visibility delta, and the histogram/transfer-function curve panel (Section 10.4) at the bottom.

`static/index.html` is a single page: a top bar (dataset picker, theme toggle, help), the viewer region (render, camera nav, per-class telemetry tiles, the curve panel), and a chat-style console (message history, a toolbar of mode toggles, and a text/voice composer). No build step — plain HTML, CSS, and vanilla JavaScript, styled with a shadcn-inspired zinc theme implemented as CSS custom properties rather than an actual shadcn/React component library (`docs/superpowers/specs/2026-09-21-minimal-shadcn-vrui-ui-design.md` made that a deliberate constraint).

`static/app.js` owns all application state and API calls. Every state-changing action funnels through `refresh(data)`, which updates the rendered image, the telemetry tiles, and (as of today) the curve panel — one place that stays in sync whenever a new step arrives, regardless of which action produced it. `static/viewer.js` wraps `vtk.js`: it streams a dataset in chunks (`/api/datasets/{name}/chunks/{i}`), reconstructs a Fortran-ordered `Float32Array`, and applies transfer-function and camera changes locally in the browser without a server round trip for every render.

10.4. The histogram/transfer-function curve panel

Added this session, directly answering the original project brief’s request for “a visual guide that plots the current transfer function on top of the histogram.” Computed entirely server-side and attached to every step (Section 10’s field table):

- `curve`: `transfer._opacity_and_color_at` sampled at the same 256 points `visibility.py`’s own lookup table uses (`visibility.lut_values()`) — a pure function of that step’s `params`, so it’s available even for a dataset with no visibility model.

- histogram: the loaded volume's own intensity histogram — the **same** 16-bin histogram `rl.oneshot_env`'s observation already carries, so the chart shows exactly what the policy sees, not a separate visualisation-only computation.

The frontend (`drawTfCurve` in `app.js`) draws the histogram sqrt-scaled (a raw CT histogram is dominated by one huge air/background bin that would otherwise flatten every tissue peak to invisible), the opacity curve traced over it, a colour ramp along the bottom built from the same RGB samples, and tick marks at the four anatomical peak centres in each peak's **actual** transfer-function colour (`transfer.ANATOMICAL_COLOURS`) — not the telemetry tiles' own accent hues, which are a UI-only palette and would otherwise contradict the ramp directly beneath them.

10.5. Voice

`asr.py` wraps `faster-whisper`; the mic button holds to record (push to talk, or Space), `/api/transcribe` returns text, and that text enters the exact same `/api/command` path a typed instruction would. There is no separate “voice mode” in the backend — voice is a text source, nothing more.

11. Preference collection and RLHF

The eventual reward-model stage this system is built towards: `/collect` shows one instruction and two candidate results (six views each), blind — no indication of which arm produced which. A rater presses A, B, equal, or skip. Roughly one item in five is drawn from a fixed anchor pool (`rl.candidates.anchor_items`) — the same items, same order, for every rater — so cross-rater agreement is measurable; the rest are sampled fresh. Roughly one item in ten silently repeats an earlier one with sides swapped, measuring a single rater’s own self-consistency, the ceiling any reward model trained on their judgments can reach.

```
python -m tools.merge_preferences out/*.jsonl --out out/preferences_merged.jsonl
```

merges multiple raters’ files (de-duplicated on pair + rater, latest timestamp wins) and reports raw agreement plus linearly-weighted Cohen’s kappa per rater pair on shared anchor items, and Krippendorff’s alpha across all raters together.

Current status: 0 clean preference judgments exist. 54 were collected before the colour-decode bug (Section 7) was fixed and are archived, not used, because a rater could tell the policy’s renders apart by their greyness alone — not a blind comparison. Reward-model training and RLHF have not started.

12. Setup and reproduction

```
python3 -m venv .venv && source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pytest -q -m "not slow"      # fast suite, 678 passing as of today
python -m pytest -q                    # everything
python server.py                       # http://127.0.0.1:8000
```

Run everything from the repository root — the code uses relative paths such as `out/` and `static/`. LLM parsing needs Ollama running locally (`ollama serve && ollama pull qwen2.5:7b`); without it, every LLM request falls back to the rule parser automatically. First voice use downloads and caches a Whisper model. `docs/REPRODUCE.md` has every command from raw data to the held-out table with real timings.

13. Reading further

- `docs/rl-paper.typ` — the thesis argument: problem, method, results, and the measurement incidents, written to be read start to end.
- `docs/STATUS.md` — the living “where things stand” document, dated, rewritten whenever something material changes.
- `docs/REPRODUCE.md` — every command from raw data to the held-out table.
- `README.md` — the short version, for anyone who wants the claim and a demo in two minutes.
- `COMMANDS.md` — the full instruction grammar, generated from `commands.COMMAND_REFERENCE` (python `-m tools.gen_commands_doc`).